

Travaux Diriger Programmation II Langage C



SMI S4

Pr AIT DAOUD

Pr OUNACER

EXERCICE 1

On veut réaliser un dictionnaire Français → Anglais. Pour cela, on utilise un tableau d'enregistrements : chaque enregistrement est formé de deux champs l'un contenant le mot en Français, l'autre la traduction en anglais.

Français	Anglais
Chat	Cat
Sac	Bag
Stylo	pen

Construire un programme permettant de:

1. Saisir ce dictionnaire
2. Supprimer un mot Français (bien sur sa traduction en anglais aussi)
3. Demander une traduction Français → Anglais
4. Demander une traduction Anglais → Français
5. Afficher la liste des mots du dictionnaire
6. Trier le dictionnaire

Solution

- **1ère méthode**

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct dictionnaire
{
    char mot[20];
    char traduction[20];
}dictionnaire;

main()
{ int n,i,j;
  int pos=-1;
  char mot[20];
  do{
    printf("donner le nombre de mots ");
    scanf("%d",&n);
  }while(n<1);
  dictionnaire *liste=(dictionnaire*)malloc(n*sizeof(dictionnaire));
  // remplissage du dictionnaire
  for(i=0;i<n;i++)
  {
    printf("le mot ");
    scanf("%s",liste[i].mot);
    printf("la traduction ");
    scanf("%s",liste[i].traduction);
  }
  printf("Français\tAnglais\n");
  printf("\n\n");
  for(i=0;i<n;i++)
  {
    printf("%s\t%s\n",(*(liste+i)).mot,(*(liste+i)).traduction);
  }
  //suppression du mot
  printf("Donner le mot a supprimer :");
```

```

scanf("%s",&mot);
for(i=0;i<n;i++)
if(strcmp((*liste+i)).mot,mot)==0){
pos=i;}

if(pos!=-1){
for(i=pos;i<n;i++){
(*liste+i)=(*liste+i+1));
}
n--;
}
//AFFICHAGE DU DICTIONNAIRE
for(i=0;i<n;i++)

printf("%s\t\t%s\n",(*liste+i)).mot,(*liste+i).traduction);

```

//traduction en français

```

printf("Entrer le mot a traduire en anglais: ");
scanf("%s",&mot);
for(i=0;i<n;i++)
if(strcmp((*liste+i)).mot,mot)==0
printf("%s\n",(*liste+i).traduction);

```

//traduction en anglais

```

printf("Entrer le mot a traduire en français: ");
scanf("%s",&mot);
for(i=0;i<n;i++)
if(strcmp((*liste+i).traduction,mot)==0)
printf("%s\n",(*liste+i).mot);

```

//tri

```

dictionnaire tmp;
for(i=0;i<n-1;i++)
for(j=i+1;j<n;j++){
if(strcmp((*liste+i)).mot,(*liste+j)).mot>0)
{
tmp=(*liste+i);
(*liste+i)=(*liste+j);
(*liste+j)=tmp;
}
}

```

//AFFICHAGE DU DICTIONNAIRE triée

```

printf("Dictionnaire triée\n");
for(i=0;i<n;i++)

printf("%s\t\t%s\n",(*liste+i)).mot,(*liste+i).traduction);
}

```

- 2 -ème méthode (en utilisant les fonctions)

```

#include <stdio.h>
#include <stdlib.h>

```

```

struct enregistrement
{
char mot[20];
char traduction[20];
};

```

```

void saisie(struct enregistrement *TabSTRUCT, int n)
{

```

```
int i;
for (i = 0; i < n; i++)
{
    printf("le mot : ");
    scanf("%s", TabSTRUCT[i].mot);
    printf("la traduction : ");
    scanf("%s", TabSTRUCT[i].traduction);
}
}

void affiche(struct enregistrement *TabSTRUCT, int n)
{
    int i;
    for (i = 0; i < n; i++)
        printf("-> %s : %s\n", TabSTRUCT[i].mot, TabSTRUCT[i].traduction);
}

int supprimer(struct enregistrement *TabSTRUCT, int n)
{
    int i, pos = -1; char mot[20];

    printf("entre le mot a supprimer : ");
    scanf("%s", &mot);

    // printf("%s ----- %s --- %d\n", mot, TabSTRUCT[1].mot, strcmp(TabSTRUCT[1].mot, mot));

    for (i = 0; i < n; i++)
        if(strcmp(TabSTRUCT[i].mot, mot) == 0) {
            pos = i;
        }
        if(pos != -1) {
            for (i = pos; i < n; i++) {
                TabSTRUCT[i] = TabSTRUCT[i+1];
            }
        }

    return pos;
}

void tfa(struct enregistrement *TabSTRUCT, int n){

    int i; char mot[20];

    printf("entre le mot a traduire en anglais : ");
    scanf("%s", &mot);

    for (i = 0; i < n; i++)
        if(strcmp(TabSTRUCT[i].mot, mot) == 0)
            printf("%s\n", TabSTRUCT[i].traduction);
}

void taf(struct enregistrement *TabSTRUCT, int n){

    int i; char mot[20];
```

```
printf("entre le mot a traduire en francais: ");
scanf("%s", &mot);
```

```
for (i = 0; i < n; i++)
if(strcmp(TabSTRUCT[i].traduction, mot) == 0)
printf("%s\n", TabSTRUCT[i].mot);
```

```
}
```

```
void tri(struct enregistrement *TabSTRUCT, int n)
{
    int i, j;
    struct enregistrement tmp;
    for (i = 0; i < n-1; ++i)
    {
        for (j = i + 1; j < n; ++j)
        {
            if(strcmp(TabSTRUCT[i].mot, TabSTRUCT[j].mot) > 0){
```

```
tmp = TabSTRUCT[i];
```

```
TabSTRUCT[i] = TabSTRUCT[j];
```

```
TabSTRUCT[j] = tmp;
```

```
}
```

```
}
```

```
}
```

```
}
```

```
void main()
{
```

```
int n;
printf("le nombre de mots : ");
scanf("%d", &n);
```

```
// dynamically allocate memory for integer TabSTRUCT of size n
struct enregistrement *TabSTRUCT = malloc(n * (sizeof(struct enregistrement)));
```

```
saisie(TabSTRUCT, n);
```

```
affiche(TabSTRUCT, n);
tfa(TabSTRUCT,n);
taf(TabSTRUCT,n);
```

```
//code pour supprimer
if(supprimer(TabSTRUCT, n) != -1) n--;
```

```
tri(TabSTRUCT, n);
```

```
affiche(TabSTRUCT, n);
```

```
}
```

EXERCICE 2

On veut réaliser un annuaire. Pour cela, on utilise une liste d'enregistrements : chaque enregistrement est formé de deux champs propres l'un contenant le nom de l'abonné, l'autre le numéro de tel correspondant.

Nom	Tel
Bennani	06150 00 40
Touilli	070 50 62 14
Alami	061 78 00 50

Construire un programme permettant de:

1. Saisir cet annuaire
2. Afficher le numéro de tel d'un abonné
3. Afficher la liste des abonnés
4. Supprimer un abonné
5. Trier l'annuaire

Solution

- **1ère méthode**

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct annuaire
{
    char nom[20];
    char numtel[20];
}annuaire;

main()
{
    int i,n,j,pos = -1;
    char nom[20];

    printf("le nombre de personnes : ");
    scanf("%d", &n);
    annuaire *liste = malloc(n * (sizeof(annuaire)));
    for (i = 0; i < n; i++)
    {
        printf("le nom : ");
        scanf("%s",liste[i].nom);
        printf("le numero : ");
        scanf("%s",liste[i].numtel);
    }
    printf("Nom\tTelephone\n\n");
    for (i = 0; i < n; i++)

        printf("%s\t%s\n", (*(liste+i)).nom, (*(liste+i)).numtel);

    printf("entre le nom à supprimer : ");
    scanf("%s", &nom);

    for (i = 0; i < n; i++)
```

```

if(strcmp((*liste+i)).nom, nom) == 0) {
    pos = i;
}
if(pos != -1) {
    for (i = pos; i < n; i++) {
        ((*liste+i)) = ((*liste+i+1));
    }
    n--;
}
printf("entre le nom a rechercher: ");
scanf("%s", &nom);

for (i = 0; i < n; i++)
if(strcmp((*liste+i)).nom, nom) == 0)
printf("%s\n", ((*liste+i)).numtel);

annuaire tmp;
for (i = 0; i < n; ++i)
{
    for (j = i + 1; j < n; ++j)
    {
        if(strcmp((*liste+i)).nom, ((*liste+j)).nom) > 0){

tmp = ((*liste+i));

(*liste+i) = ((*liste+j));

(*liste+j) = tmp;
        }
    }
    printf("Annuaire triee\n");
for (i = 0; i < n; i++)
    printf("-> %s : %s\n", ((*liste+i)).nom, ((*liste+i)).numtel);
}

```

- 2 -ème méthode (en utilisant les fonctions)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct annuaire Annuaire;
struct annuaire
{
    char nom[100];
    char telephone[100];
};

void saisie(Annuaire *TabSTRUCT, int n)
{
    int i;
    for (i = 0; i < n; i++)
    {
        printf("le nom : ");
    }
}

```

```
scanf("%s", TabSTRUCT[i].nom);
printf("le telephone : ");
scanf("%s", TabSTRUCT[i].telephone);
}
}

void affiche(Annuaire *TabSTRUCT, int n)
{
int i;
for (i = 0; i < n; i++)
printf("-> %s : %s\n", TabSTRUCT[i].nom, TabSTRUCT[i].telephone);
}

void recherche(Annuaire *TabSTRUCT, int n)
{
int i, pos = -1;
char nom[20];

printf("entre le nom a rechercher: ");
scanf("%s", &nom);

for (i = 0; i < n; i++)
if(strcmp(TabSTRUCT[i].nom, nom) == 0)
printf("%s\n", TabSTRUCT[i].telephone);
}

int supprimer(Annuaire *TabSTRUCT, int n)
{
int i, pos = -1;
char nom[20];
// printf("%s ----- %s --- %d\n", nom, TabSTRUCT[1].nom, strcmp(TabSTRUCT[1].nom, nom));

printf("entre le nom a supprimer : ");
scanf("%s", &nom);
for (i = 0; i < n; i++)
if(strcmp(TabSTRUCT[i].nom, nom) == 0) {
pos = i;
}
if(pos != -1) {
for (i = pos; i < n; i++) {
TabSTRUCT[i] = TabSTRUCT[i+1];
}
n--;
}

return pos;
}

void tri(Annuaire *TabSTRUCT, int n)
{
int i, j;
Annuaire tmp;
for (i = 0; i < n-1; ++i)
{
for (j = i + 1; j < n; ++j)
{
```



```
        if(strcmp(TabSTRUCT[i].nom, TabSTRUCT[j].nom) > 0){

tmp = TabSTRUCT[i];

TabSTRUCT[i] = TabSTRUCT[j];

TabSTRUCT[j] = tmp;
    }
    }
}

void main()
{

int n;
printf("saisir le nombre d'enregistrement que vous voulez entrer : ");
scanf("%d", &n);

// dynamically allocate memory for integer TabSTRUCT of size n
Annuaire *TabSTRUCT = malloc(n * (sizeof(Annuaire)));

//Saisir les enregistrement dans l'annuaire
saisie(TabSTRUCT, n);

//afficher la liste des abonnés
affiche(TabSTRUCT, n);
//chercher le numéro d'un abonné
recherche(TabSTRUCT, n);

//code pour supprimer
if(supprimer(TabSTRUCT, n) != -1) n--;

//trier la liste des abonnés
tri(TabSTRUCT, n);
//afficher la liste des abonnés
affiche(TabSTRUCT, n);

}
```

EXERCICE 3

Écrire un programme qui permet de créer une structure Point qui permet de stocker les coordonnées des points au fur et à mesure les uns au-dessus des autres sur un écran.

Initialisation()

Empilage()

Dépilage()

Affichage()

Solution

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct Point Point;
```

```
struct Point
```

```
{
```

```
    int x;
```

```
    int y;
```

```
    Point *suivant;
```

```
};
```

```
typedef struct Pile Pile;
```

```
struct Pile
```

```
{
```

```
    Point *premier;
```

```
};
```

```
Pile *initialisation()
```

```
{
```

```
    Pile *pile = malloc(sizeof(*pile));
```

```
    Point *point = malloc(sizeof(*point));
```

```
    if (pile == NULL || point == NULL)
```

```
    {
```

```
        exit(EXIT_FAILURE);
```

```
    }
```

```
    point->x = 0;
```

```
    point->y = 0;
```

```
    point->suivant = NULL;
```

```
    pile->premier = point;
```

```
    return pile;
```

```
}
```

```
void empiler(Pile *pile, int nvx, int nvy)
```

```
{
```

```
    Point *nouveau = malloc(sizeof(*nouveau));
```

```
    if (pile == NULL || nouveau == NULL)
```

```
    {
```

```
        exit(0);
```

```
    }
```

```
    nouveau->x = nvx;
```

```
    nouveau->y = nvy;
```

```
    nouveau->suivant = pile->premier;
```

```
    pile->premier = nouveau;
```

```
}
```

```
int depiler(Pile *pile)
{
    if (pile == NULL)
    {
        exit(EXIT_FAILURE);
    }
    int xDepile = 0, yDepile = 0;
    Point *PointDepile = pile->premier;

    if (pile != NULL && pile->premier != NULL)
    {
        xDepile = PointDepile->x;
        yDepile = PointDepile->y;
        pile->premier = PointDepile->suivant;
        free(PointDepile);
    }
    return xDepile;
}

void afficherPile(Pile *pile)
{
    if (pile == NULL)
    {
        exit(EXIT_FAILURE);
    }
    Point *actuel = pile->premier;

    while (actuel != NULL)
    {
        printf("X:%d.....Y:%d\n", actuel->x, actuel->y);
        actuel = actuel->suivant;
    }
    printf("\n");
}

int main()
{
    Pile *maPile = initialisation();

    empiler(maPile, 9, 5);
    empiler(maPile, 24, 9);
    empiler(maPile, 10, 2);
    empiler(maPile, 6, 10);
    empiler(maPile, 3, 71);
    empiler(maPile, 8, 34);

    printf("\nEtat de la pile :\n");
    afficherPile(maPile);

    printf("Je depile %d\n", depiler(maPile));
    printf("Je depile %d\n", depiler(maPile));

    printf("\nEtat de la pile :\n");
    afficherPile(maPile);

    return 0;
}
```